

C r e a t o r

Creator Principle

Who instantiates whom

DEFINITION

Let class B create A when B aggregates, contains, records, or closely uses A.

Creator answers a narrow but constant question: who should call new? GRASP gives concrete criteria — let B instantiate A when B aggregates, contains, records, or closely uses A, or already holds the data A needs to be initialised. The object that will own or use the new instance is usually the one with everything required to build it.

WHY IT MATTERS

Assigning creation to the natural owner avoids inventing a separate creator and adds no new dependency: B already references A's type, so making B build A introduces no extra coupling. It also keeps lifecycles honest — the thing responsible for an object's existence is the thing that already manages it.

— How it works

Creator	The class that instantiates the new object — chosen because it aggregates, contains, records, or closely uses it.
Created	The object being constructed and, typically, owned or used by the Creator.
Criteria	The test for who creates: containment / aggregation is the strongest signal; "has the initialising data" comes next.

HOW TO APPLY

1. Identify the new object and who will hold or use it.
2. Check the criteria: does a class contain, aggregate, record, or closely use it?
3. If yes, give that class the creation responsibility.
4. If construction is complex or conditional, delegate to a Factory or Builder instead.

LITMUS TEST

Does this class contain, aggregate, or closely use the new object? If yes, it is the natural creator. If construction is complex or conditional, delegate to a Factory or Builder instead.

— Smell → fix

A service builds the Line that only Cart ever holds — needless coupling to Cart's internals:

```
class Cart { lines: Line[] = [] }
class CartService {
  addItem(cart: Cart, p: Product) { // outsider builds it
    cart.lines.push(new Line(p))
  }
}
```

↓ LET THE OWNER CREATE IT

```
class Cart {
  lines: Line[] = []
  add(p: Product) { // Cart owns Lines, so it creates them
    this.lines.push(new Line(p))
  }
}
```

Cart already aggregates Lines, so it owns their creation. No service or factory is needed until construction grows complex — then delegate to a Builder or Factory.

USE WHEN

- ✓ Deciding which class calls new
- ✓ B already holds or closely uses A

AVOID WHEN

- ▲ Complex construction — delegate to Factory Method / Builder

— Trade-offs & pitfalls

PROS	CONS
+ Low coupling — no extra creator class + Natural ownership	– Poor fit when creation logic is complex

COMMON PITFALLS

- ▲ Reaching for a Factory by default — for simple objects the owning class is the lower-coupling creator.
- ▲ Putting creation on a class that neither owns nor uses the object, adding a dependency for no reason.
- ▲ Hiding complex, branchy construction inside an aggregate where a Builder or Factory would read far better.

WHERE YOU SEE IT

- `cart.add(p)` building the `Line` it will hold, rather than a `CartService` newing it from outside.
- An `Order` creating its `OrderItems`; a `Document` creating its `Paragraphs`.
- A `Factory` taking over once construction needs polymorphism or many parameters.

SEE ALSO

Factory Method	Creator says who instantiates; Factory Method is how, when creation needs subtype variation
Builder	delegate here when construction is complex or multi-step
Information Expert	sibling heuristic — Expert places behaviour, Creator places construction
Low Coupling	creating where ownership lives adds no new dependency

— Interview & sources

Creator (GRASP) vs Factory Method (GoF)?

Creator is a principle for deciding who instantiates; Factory Method is a pattern that defers instantiation to subclasses. Creator often tells you where a factory belongs.

What are the Creator criteria?

B should create A if B aggregates, contains, records, or closely uses A — or holds A's initialising data. The strongest signal is aggregation or containment.

When should I ignore Creator?

When construction is complex, conditional, or drags in many dependencies — then a Factory or Builder gives better cohesion than the container.

Creator vs Information Expert — how do they relate?

Two sides of responsibility assignment: Expert places behaviour with the data, Creator places instantiation with the owner. Both push work toward the class that already has the relevant state.

REMEMBER

Whoever owns or closely uses an object should be the one to create it.

SOURCES

Craig Larman — "Applying UML and Patterns" (3rd ed., 2004): Creator

GoF — "Design Patterns" (1994): Factory Method and Builder for delegated creation